AI Studio

Code Review with Coding Agents

Lecture notes

Why rethink code review

- Goal: **optimise code review for human time and effort**.
- With agents writing the code, reviewing everything manually becomes almost impossible.

Code review serves two purposes:

(A) Detect and correct bugs.

(B) Develop a shared understanding of the codebase.

- Traditionally, review happens *after the fact*: you wait for someone to take a look.
- With agents, code review is integrated *into the workflow*, like a spell-check that runs continuously.
- This catches almost all the routine bugs (A) and saves human attention for (B) and for higher-level considerations.

Note: if review time goes down but bugs still occur and take time to fix, the problem has not been solved.

The Claude Code SWEs' workflow

Legend: needs a spending cap / frequency setting *my own annotation*

1 · Before the PR

i) Claude verifies the work by running the tests and the linter, and is asked to show the results.

ii) Another agent with a fresh context reviews the diff. *I would not include this*

iii) Required CI: lint, types, tests across platforms, build and packaging.
⇒ *open findings block the merge.*

iv) Agents hunt for bugs and review in parallel, against the knowledge base. spending cap

v) Verifiers vote on each finding; the survivors are ranked by severity. spending cap
⇒ *open findings block the merge.*

vi) A separate security review. *optional* spending cap

⇒ **Risk-based approval.** A classifier sets the tier:

no human review | fix PR (up to 2 rounds) | human review

2 · Merging

i) Checks run again against the current `develop` and feed the merge queue. spending cap

3 · After the merge

i) Monitor for bugs and post-hoc findings ⇒ an issue creator opens fix-forwards or reverts.
set monitoring frequency

ii) A fast rollback mechanism.

Result: few bugs get through all the layers, and when some do, a fast rollback is in place.

Instruction files

- Keep `CLAUDE.md` and `AGENTS.md` **brief**, and have them reference the rest of the docs.
- `REVIEW.md`: here each team can specify what matters in their part of the code:
  - define what counts as a failure for the project;
  - limit nitpicking;
  - define ignored paths (could be updated or moved to the knowledge base);
  - require a file + line reference for every finding;
  - no guessing of file names;
  - require a top-level summary;
  - always try to find the root cause.

  Important: flag pre-existing bugs as *nits*, so the author is not unfairly penalised for bugs the PR did not introduce.
- `VOTERS.md`: shown **only to the verifiers**, so that it cannot be gamed.
  - Findings about style, diagnostics and cleanups are filtered out. (A/B tested by Anthropic)
  - Other rules come from `AGENTS.md`.

Risk tiers and process

Risk tiers are influenced by the file type:

Kind of change	Review
Docs, copy, additive changes to code that is already implemented	No human
Ordinary changes, sensitive changes, system prompts, user-defined	Human

- Always put `feat/`, `bugfix/` and `flags` on PRs.
- Collect incidents and use them to update the evaluation rubrics.
- Stated priorities: **(1)** detect real bugs; **(2)** reduce the number of review cycles.

Where humans in the loop are needed

- Deciding whether or not to build something.
- Setting the vision, the taste and the overall goals of the product.
- Auditing risky code, for purpose (A).
- Verifying the outputs (the end product).

About (B): understanding the codebase

If machines do the bulk of the review and diff reading, how do people learn the codebase? Do they even need to?

> Anthropic SWEs still do not have a formal mechanism for this.

- Nobody needs a full mental model. When someone wants to learn about a piece of the codebase, they ask Claude to explain it.
- The coding workflow becomes:

